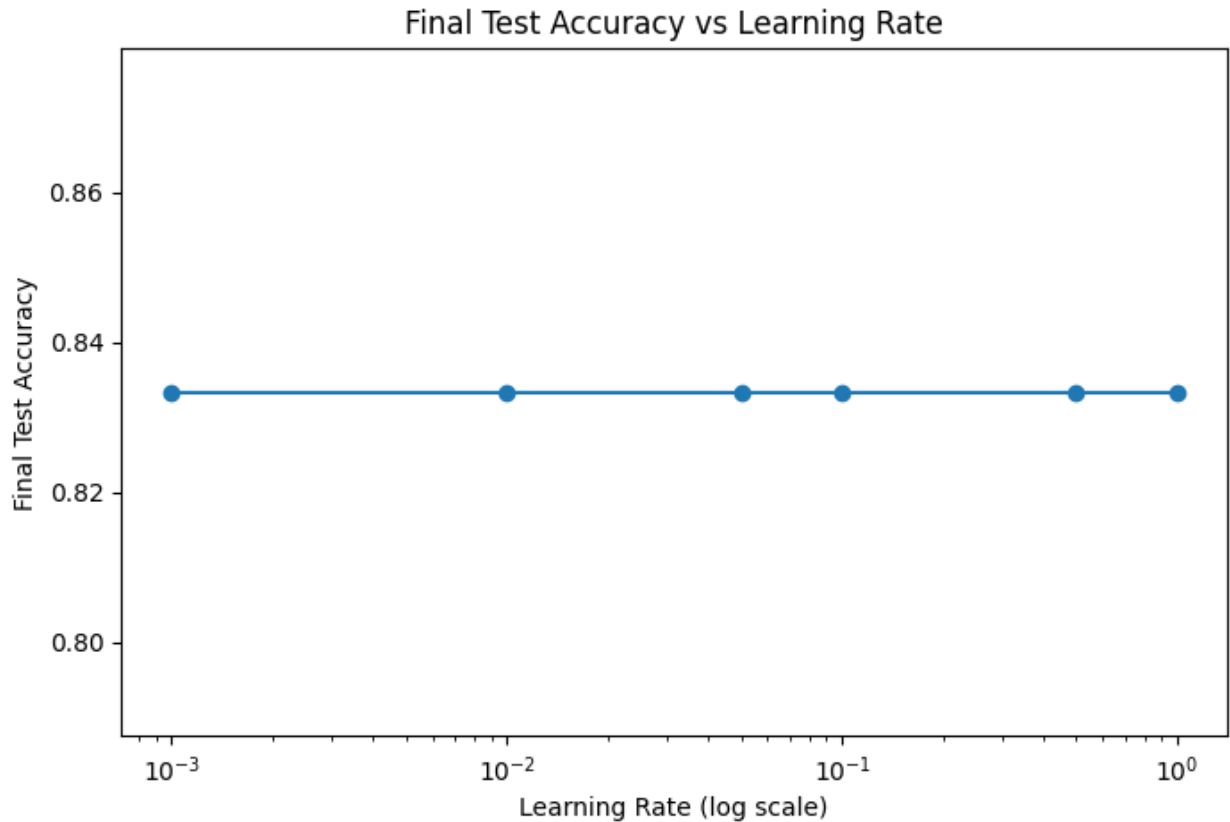




Final Test Accuracy for each LR:
LR=0.001 -> Test Accuracy=0.8333
LR=0.01 -> Test Accuracy=0.8333
LR=0.05 -> Test Accuracy=0.8333
LR=0.1 -> Test Accuracy=0.8333
LR=0.5 -> Test Accuracy=0.8333
LR=1.0 -> Test Accuracy=0.8333

LAB 5

```
import numpy as np

# XOR dataset (4 input combinations)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# XOR output
y = np.array([
```

```

        [0],
        [1],
        [1],
        [0]
    ])

# Sigmoid activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Set random seed for same results
np.random.seed(42)

# Network architecture
input_neurons = 2
hidden_neurons = 2
output_neurons = 1

# Initialize weights and biases
W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.random.randn(1, hidden_neurons)

W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.random.randn(1, output_neurons)

# Hyperparameters
learning_rate = 0.1
epochs = 10000

# Training loop
for epoch in range(epochs):

    # ----- Forward Propagation -----
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)

    loss = np.mean((y - final_output) ** 2)

    # ----- Backpropagation -----
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(W2.T)

```

```

d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# ----- Update Weights and Biases -----
W2 += hidden_output.T.dot(d_output) * learning_rate
b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

W1 += X.T.dot(d_hidden) * learning_rate
b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

if epoch % 1000 == 0:
    print(f"Epoch {epoch}, Loss: {loss:.6f}")

print("\nFinal Predictions:")
for i in range(len(X)):
    prediction = final_output[i][0]
    predicted_class = 1 if prediction >= 0.5 else 0
    print(f"Input: {X[i]} -> Predicted: {prediction:.4f} -> Class: {predicted_class}")

Epoch 0, Loss: 0.294351
Epoch 1000, Loss: 0.244410
Epoch 2000, Loss: 0.203539
Epoch 3000, Loss: 0.153349
Epoch 4000, Loss: 0.046331
Epoch 5000, Loss: 0.015615
Epoch 6000, Loss: 0.008448
Epoch 7000, Loss: 0.005614
Epoch 8000, Loss: 0.004146
Epoch 9000, Loss: 0.003264

Final Predictions:
Input: [0 0] -> Predicted: 0.0540 -> Class: 0
Input: [0 1] -> Predicted: 0.9505 -> Class: 1
Input: [1 0] -> Predicted: 0.9501 -> Class: 1
Input: [1 1] -> Predicted: 0.0536 -> Class: 0

```

LAB 8| 12 March 2026

Change the batch size

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Load dataset

```